

Egor Klimov
Team Lead

Nikita Dukin
Researcher

Saga Rut Sunnevdóttir
Researcher

Why do Agents Fail?

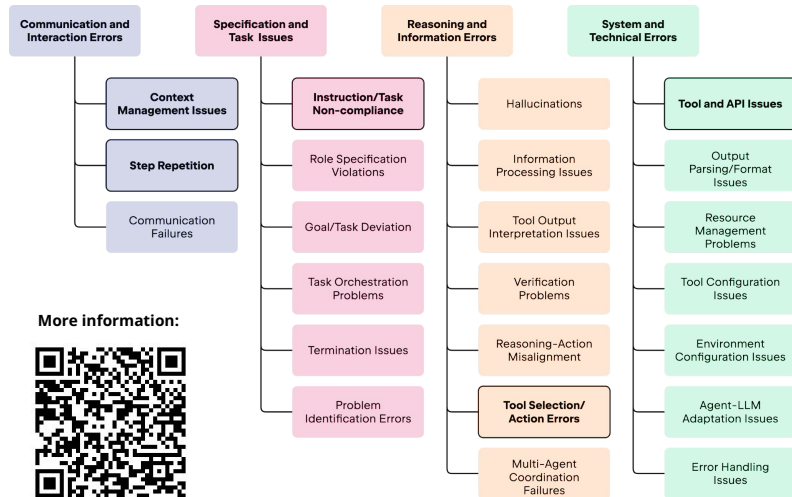
While LLM agents are powerful, they often fail in non-trivial ways. Evaluations commonly focus on success rates, offering little insight into why agents fail. Understanding this is essential for improving reliability and guiding development of agents.

To start with, we collected a taxonomy of common agent failures.

Literature Survey + Team interviews
= Combined Taxonomy

Interview Insights:

- Focus on tool call patterns and reliability as they are key aspects of agents
- A single trace is insufficient -> study multiple runs per task
- Exceptions or status alone don't show success -> check evaluation results



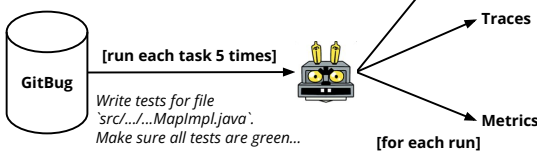
More information:



Execution Dataset

To analyze agent behavior, we used the **Test Generation Agent** due to their established evaluation pipeline (check out their poster tool):

- Repeated the same task multiple times
- Collected and correlated logs, traces, and eval metrics
- Applied analyzers to identify issues



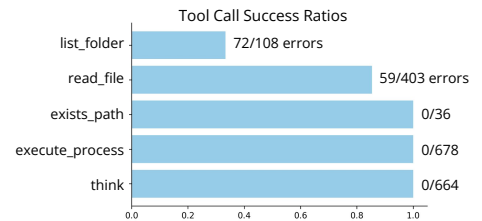
```
{
  "@timestamp": "2025-09-12T15:36:28.325581+02:00",
  "message": "Finished executing node...",
  "level": "DEBUG",
  "trace_id": "990c80661276f121b8bfa0cb0592f1d",
  "span_id": "9c887c2b21ffe2ad"
}

{
  "traceID": "990c80661276f121b8bfa0cb0592f1d",
  "spanID": "9c887c2b21ffe2ad",
  "operationName": "node.callTool",
  "duration": 4241,
  "tags": [...]
}

"execution_based_metrics": {
  "...Test.java": {
    "line_coverage": 0.5769230769230769,
    "branch_coverage": 0.25
  }
}
```

Initial Insights

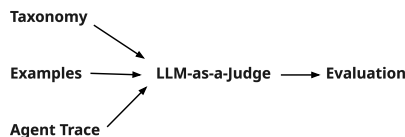
From analysis of the dataset:



- Additional notes:
 - High correlation between std dev of number of steps and success rate
 - No significant correlation between coverage and number of steps
 - Mismatch cases (agent vs. evaluation disagreement)

LLM-as-a-judge

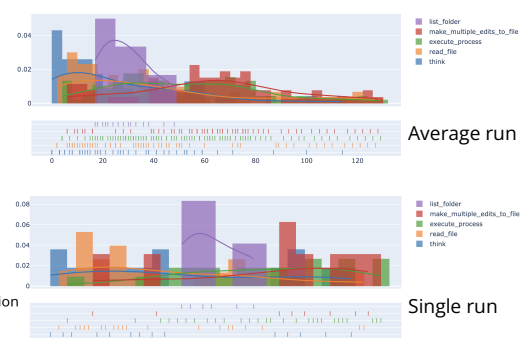
Reimplemented MAST LLM-as-a-judge from *Why Do Multi-Agent Systems Fail?*



- Hard to verify results
- Not tuned to our data
- Quick results
- Semantic analysis

Tool call distribution

- Aggregate tool calls into time-binned histograms over N runs
- Compare single vs. average with Wasserstein distance
- Deviations is a signal to suspicious execution



- Hard to know ideal N
- Different success paths
- Tool call analysis
- Outlier run detection

FUTURE WORK



Refined LLM-as-a-judge

- Tune to our dataset -> adapt prompts, taxonomy, and examples
- Distill richer context -> combine logs, traces, and metrics for input
- Expand evaluation -> improve dataset and its coverage across failure types
- Integration with Langfuse



Refined tool call distribution analysis

- Introduce patterns of solving a task, based on tool distribution
- Classify runs by pattern (e.g., success with tool N, success with tool M)
- Explore how different tool calls patterns affect an average run